

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по рубежному контролю №2
Вариант Д7



Выполнила:
Студентка группы ИУ5-36Б
Крахмальникова А. И.

Проверил:
Гапанюк Ю. Е.

Москва 2025

Листинг кода

RK1_refactored.py

```
from operator import itemgetter

class Microprocessor:
    """Микропроцессор"""

    def __init__(self, id, model, frequency, computer_id):
        self.id = id
        self.model = model
        self.frequency = frequency
        self.computer_id = computer_id

class Computer:
    """Компьютер"""

    def __init__(self, id, name):
        self.id = id
        self.name = name

class MicroprocessorComputer:
    """Связь многие-ко-многим между микропроцессорами и компьютерами"""

    def __init__(self, computer_id, microprocessor_id):
        self.computer_id = computer_id
        self.microprocessor_id = microprocessor_id

class DataManager:
    """Класс для управления данными и выполнения запросов"""

    def __init__(self, computers=None, microprocessors=None,
microprocessors_computers=None):
        """
        Инициализация менеджера данных

        Args:
            computers: список компьютеров
            microprocessors: список микропроцессоров
            microprocessors_computers: список связей многие-ко-многим
        """
        self.computers = computers or []
        self.microprocessors = microprocessors or []
        self.microprocessors_computers = microprocessors_computers or []

    def get_one_to_many(self):
        """
        Соединение данных один-ко-многим
        """
```

```

Returns:
    список кортежей (model, frequency, computer_name)
    """
    return [(m.model, m.frequency, c.name)
            for c in self.computers
            for m in self.microprocessors
            if m.computer_id == c.id]

def get_many_to_many(self):
    """
    Соединение данных многие-ко-многим

    Returns:
        список кортежей (model, frequency, computer_name)
        """
    many_to_many_temp = [(c.name, mc.computer_id, mc.microprocessor_id)
                        for c in self.computers
                        for mc in self.microprocessors_computers
                        if c.id == mc.computer_id]

    return [(m.model, m.frequency, comp_name)
            for comp_name, comp_id, mproc_id in many_to_many_temp
            for m in self.microprocessors if m.id == mproc_id]

def task1(self, one_to_many):
    """
    Задание Д1: микропроцессоры, у которых модель заканчивается на 'n'

    Args:
        one_to_many: список соединений один-ко-многим

    Returns:
        список отфильтрованных записей
        """
    return list(filter(lambda i: i[0].endswith('n'), one_to_many))

def task2(self, one_to_many):
    """
    Задание Д2: средняя частота микропроцессоров для каждого компьютера

    Args:
        one_to_many: список соединений один-ко-многим

    Returns:
        список кортежей (computer_name, avg_frequency) отсортированный по частоте
        """
    res_unsorted = []

    for c in self.computers:
        # Список микропроцессоров компьютера
        c_microprocessors = list(filter(lambda i: i[2] == c.name, one_to_many))
        if len(c_microprocessors) > 0:
            # Частоты микропроцессоров

```

```

        c_freqs = [freq for _, freq, _ in c_microprocessors]
        # Средняя частота
        avg_freq = sum(c_freqs) / len(c_freqs)
        res_unsorted.append((c.name, round(avg_freq, 2)))

    return sorted(res_unsorted, key=itemgetter(1))

def task3(self, many_to_many):
    """
    Задание Д3: компьютеры, название которых начинается с 'A',
    со списком моделей их микропроцессоров

    Args:
        many_to_many: список соединений многие-ко-многим

    Returns:
        словарь {computer_name: [model1, model2, ...]}
    """
    result = {}

    for c in self.computers:
        if c.name.startswith('A'):
            # Список микропроцессоров компьютера (многие-ко-многим)
            c_microprocessors = list(filter(lambda i: i[2] == c.name, many_to_many))
            # Модели микропроцессоров
            c_models = [model for model, _, _ in c_microprocessors]
            result[c.name] = c_models

    return result

def get_default_data():
    """Функция для получения данных по умолчанию"""
    # Компьютеры
    computers = [
        Computer(1, 'Acer'),
        Computer(2, 'Компьютер1'),
        Computer(3, 'A server'),
        Computer(11, 'Asus'),
        Computer(22, 'Ноутбук'),
        Computer(33, 'Компьютер2'),
    ]

    # Микропроцессоры
    microprocessors = [
        Microprocessor(1, 'Intel Xeon', 3500, 1),
        Microprocessor(2, 'AMD', 3200, 2),
        Microprocessor(3, 'Intel', 2800, 3),
        Microprocessor(4, 'AMD Ryzen', 3000, 3),
        Microprocessor(5, 'Intel Pentium', 2400, 3),
    ]

    # Связи многие-ко-многим
    microprocessors_computers = [

```

```
    MicroprocessorComputer(1, 1),
    MicroprocessorComputer(2, 2),
    MicroprocessorComputer(3, 3),
    MicroprocessorComputer(3, 4),
    MicroprocessorComputer(3, 5),
    MicroprocessorComputer(11, 1),
    MicroprocessorComputer(22, 2),
    MicroprocessorComputer(33, 3),
    MicroprocessorComputer(33, 4),
    MicroprocessorComputer(33, 5),
]
```

```
return computers, microprocessors, microprocessors_computers
```

```
def main():
    """Основная функция для демонстрации работы"""
    # Получаем данные по умолчанию
    computers, microprocessors, microprocessors_computers = get_default_data()

    # Создаем менеджер данных
    manager = DataManager(computers, microprocessors, microprocessors_computers)

    # Получаем соединения
    one_to_many = manager.get_one_to_many()
    many_to_many = manager.get_many_to_many()

    # Выполняем задания
    print('Задание Д1')
    res1 = manager.task1(one_to_many)
    print(res1)

    print('\nЗадание Д2')
    res2 = manager.task2(one_to_many)
    print(res2)

    print('\nЗадание Д3')
    res3 = manager.task3(many_to_many)
    print(res3)

if __name__ == '__main__':
    main()
```

RK2.py

```
import unittest
from RK1_refactored import Computer, Microprocessor, MicroprocessorComputer, DataManager

class TestRK1(unittest.TestCase):
    """Класс для модульного тестирования программы RK1"""

    def setUp(self):
        """Подготовка тестовых данных перед каждым тестом"""
        # Тестовые данные
        self.computers = [
            Computer(1, 'Test Computer 1'),
            Computer(2, 'A Test Computer'),
            Computer(3, 'Test Computer 2'),
        ]

        # Тестовые микропроцессоры
        self.microprocessors = [
            Microprocessor(1, 'Intel Xeon', 3500, 1),
            Microprocessor(2, 'AMD Ryzen', 3200, 1),
            Microprocessor(3, 'Intel Pentium', 2800, 2),
            Microprocessor(4, 'AMD', 3000, 3),
        ]

        self.microprocessors_computers = [
            MicroprocessorComputer(1, 1),
            MicroprocessorComputer(1, 2),
            MicroprocessorComputer(2, 3),
            MicroprocessorComputer(3, 4),
        ]

        self.manager = DataManager(
            self.computers,
            self.microprocessors,
            self.microprocessors_computers
        )

    # Тест 1: Задание Д1
    def test_task1_microprocessors_ending_with_n(self):
        """Тест задания Д1: поиск микропроцессоров, чья модель заканчивается на 'n'"""
        one_to_many = self.manager.get_one_to_many()
        result = self.manager.task1(one_to_many)

        # Проверяем, что все найденные модели заканчиваются на 'n'
        for item in result:
            self.assertTrue(item[0].endswith('n'),
                             f"Модель '{item[0]}' не заканчивается на 'n'")

        # Проверяем конкретные ожидаемые результаты
        expected_models = ['Intel Xeon', 'AMD Ryzen']
        result_models = [item[0] for item in result]
```

```

# Проверяем наличие ожидаемых моделей
for model in expected_models:
    self.assertIn(model, result_models,
                   f"Модель '{model}' должна быть в результатах")

# Проверяем, что модели, НЕ заканчивающиеся на 'n', отсутствуют
unexpected_models = ['Intel Pentium', 'AMD']
for model in unexpected_models:
    self.assertNotIn(model, result_models,
                      f"Модель '{model}' не должна быть в результатах")

# Тест 2: Задание Д2
def test_task2_average_frequency_calculation(self):
    """Тест задания Д2: вычисление средней частоты микропроцессоров по компьютерам"""
    one_to_many = self.manager.get_one_to_many()
    result = self.manager.task2(one_to_many)

    # Проверяем структуру результата
    self.assertIsInstance(result, list, "Результат должен быть списком")

    for item in result:
        self.assertIsInstance(item, tuple, "Каждый элемент должен быть кортежем")
        self.assertEqual(len(item), 2, "Кортеж должен содержать 2 элемента")
        self.assertIsInstance(item[1], (int, float),
                                "Второй элемент должен быть числом")

    # Проверяем правильность вычислений для Test Computer 1
    # У него 2 процессора: Intel Xeon (3500) и AMD Ryzen (3200), среднее = 3350
    for computer_name, avg_freq in result:
        if computer_name == 'Test Computer 1':
            self.assertAlmostEqual(avg_freq, 3350.0, places=2,
                                    msg=f"Неправильная средняя частота для {computer_name}")

# Тест 3: Задание Д3
def test_task3_computers_starting_with_A(self):
    """Тест задания Д3: компьютеры, начинающиеся с 'A', со списком моделей процессоров"""
    many_to_many = self.manager.get_many_to_many()
    result = self.manager.task3(many_to_many)

    # Проверяем структуру результата
    self.assertIsInstance(result, dict, "Результат должен быть словарем")

    # Проверяем, что в результатах только компьютеры, начинающиеся с 'A'
    for computer_name in result.keys():
        self.assertTrue(computer_name.startswith('A'),
                        f"Компьютер '{computer_name}' должен начинаться с 'A'")

    # Проверяем, что 'A Test Computer' есть в результатах
    self.assertIn('A Test Computer', result,
                  "Компьютер 'A Test Computer' должен быть в результатах")

```

```
# Проверяем, что у 'A Test Computer' есть список моделей
self.assertIsInstance(result['A Test Computer'], list,
                        "Значение должно быть списком моделей")

# Проверяем конкретную модель (Intel Pentium)
self.assertIn('Intel Pentium', result['A Test Computer'],
              "Модель 'Intel Pentium' должна быть в списке")

if __name__ == '__main__':
    # Запуск модульных тестов
    unittest.main()
```


Скриншоты работы программы

